Custom UDP Server vs. Godot ENet: A Cloud Benchmark of Multiplayer Server Capacity

Carlos Nunez
Department of Computer Science and Engineering, Shippensburg University
Mentor: Dr. Girard
CSC 499 – Senior Research Methods

Fall 2026 (draft)

Abstract

Developers who build multiplayer games in Godot must choose between the engine's built-in ENet dedicated server and a custom server written outside the engine. We compare a custom C++ UDP server with Godot's ENet headless server under the same workload, NodeStrike, a 20×20 grid game in which every client is a bot that moves one tile per 20 Hz server tick. Both servers run on Google Cloud with one server VM and three client VMs. Two tests are used: a progressive ladder that adds clients from 1 to 300, run eight times per backend, and a 20-minute hold at fixed loads of 180 to 300 clients. We measure latency, packet handling (snapshots received per client per second), client limit, and server tick rate. Across the eight ladder runs the C++ server passed all 292 states every time, with a round-trip time near 6 ms. ENet passed at 100 and 150 clients in all eight runs, at 200 clients in three of eight, and at 250 clients in none; its first failure came between 170 and 188 clients in every run, with ping and lost snapshots growing beyond that. In the 20-minute holds, C++ kept every client responding at 180, 260, 270 and 300 clients in all eight runs, with ping near 6 ms. ENet held 180 clients in all eight runs, but held 260 and 270 clients in only one run each and 300 clients in none; at 300 clients about 75% of its clients responded and ping was near 1.4 s. ENet's server tick rate stayed near 20 Hz throughout, so tick rate alone did not reveal its overload. Both tests are the mean of eight complete runs per backend.

Keywords: game networking, UDP, ENet, Godot, tick rate, server capacity, benchmarking

1 Introduction

TODO: write the introduction (problem, what this paper does, the questions).

2 Background and Related Work

TODO: write the background (servers and transport, related work).

2.1 Ticks and snapshots

TODO: write what a tick and a snapshot are.

Term	Meaning in NodeStrike
Hz (hertz)	Times per second. NodeStrike runs at 20 Hz, i.e. 20 ticks per second.
Tick	One server cycle: read incoming client updates, update the world, send a snapshot. Fixed at 50 ms (1000/20).
Snapshot	A capture of the game state at one tick: every player's position at that instant. The server sends it to each client ($16 + 12N$ bytes for N players, split into datagrams of at most 1,200 bytes). Each arrival updates the client's current ping reading and gap count.
Latency / ping	How long data takes to travel. Round-trip time (RTT) is there and back. In the game client, ping is the time from sending its last position update until the next snapshot arrives, so it includes waiting for the server's next 50 ms tick.
Gap / loss	Each snapshot has a sequence number; a missing number means a snapshot was lost or dropped.
Effective tick rate	Ticks the server actually completes per second. Healthy $\approx$ 20 Hz; below 18 Hz is treated as overloaded.

Table 1: Terms used in this paper.

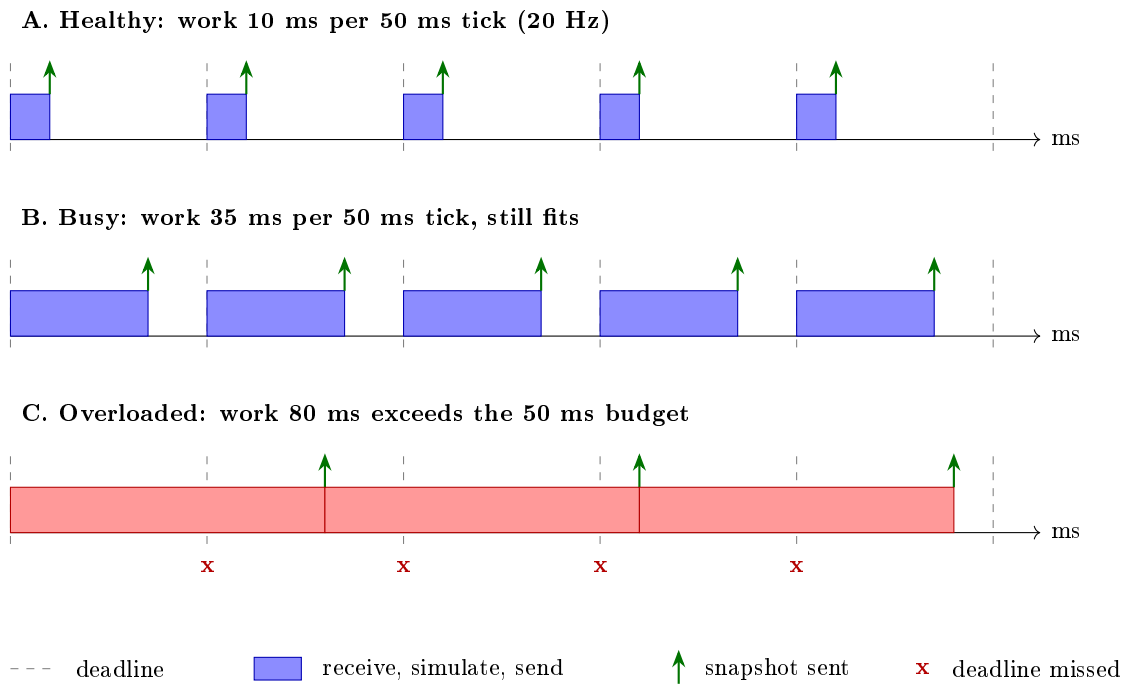


Figure 1: Schematic of a server tick (illustrative timings, not measured data). Each tick the server receives client input, updates the world, and sends a snapshot; at 20 Hz it has 50 ms for all of it. If the work per tick grows with the number of clients and passes 50 ms, ticks start late, the effective tick rate falls ($1000/80 = 12.5$ Hz in panel C) and clients get fewer, later snapshots.

3 Making of the Godot Client and ENet Server

TODO: write how the Godot client and the Godot ENet server were built (goals, design decisions, problems you hit).

Notes from the code (delete when written).

- One Godot 4.7 project, NodeStrike (20×20 grid). One script, `main.gd` (about 860 lines), runs every role.
- The role is chosen with command-line flags: `-transport=udp|enet`, `-role=client|server`, `-address`, `-port`, `-udp-only`, `-enet-snapshot=binary|variant`.
- Client: spawns a bot, picks a goal tile, moves one tile per tick from a 20 Hz timer, and sends its position every tick (an 18-byte message over UDP, or a remote procedure call over ENet).
- Client default is the C++ UDP server; a windowed client falls back to ENet after 8 s with no reply, unless `-udp-only` is set.
- ENet server mode: creates an ENet multiplayer peer (up to 2048 clients), runs a 20 Hz timer, drops clients silent for more than 5 s, and each tick sends one snapshot to every peer by remote procedure call (compact binary, or a list of dictionaries).
- Headless mode (no window) is used for the benchmark and prints ping, packets sent and received, and gaps in snapshot numbers.
- Two small extra scripts, `enet_server.gd` and `enet_client.gd`, are a simpler ENet pair that the cloud benchmark does not use.

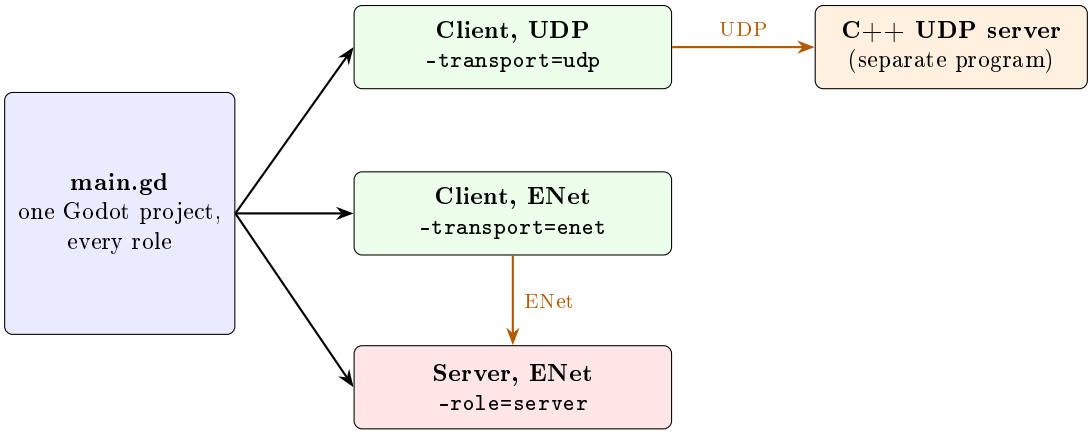


Figure 2: The Godot project. One script is started in different modes. A UDP client talks to the separate C++ server; an ENet client talks to the same project started as an ENet server.

4 Making of the C++ UDP Server

TODO: write how the C++ server was built (goals, design decisions, problems you hit).

Notes from the code (delete when written).

- One file, `server/main.cpp` (about 590 lines), C++17, plain POSIX sockets, one thread.
- Binary protocol, version 2 (Table 2). A player is identified by its network address, not by anything the client sends.

- Main loop (Figure 3): wait up to 5 ms for a packet, read everything waiting, and when 50 ms have passed since the last tick, run a tick.
- A tick drops clients silent for more than 5 s and sends a snapshot to every client, split into datagrams of at most 1,200 bytes.
- Once a second it prints one telemetry line (tick number, players, packets and bytes in and out); the benchmark records these lines.
- A separate terminal display, `tui/tui.cpp`, exists for watching a server.

Message	Direction	Size	Contents
JOIN	client to server	2 B	type, version
ACK	server to client	6 B	type, version, player id
STATE	client to server	18 B	position, timestamp, number, goal
PING	both ways	10 B	client timestamp, number (echoed back)
TICK (snapshot)	server to client	16 + 12N B	header, then id, position and goal for each of N players

Table 2: Messages of the C++ server protocol (version 2). Snapshots larger than 1,200 bytes are split into several datagrams.

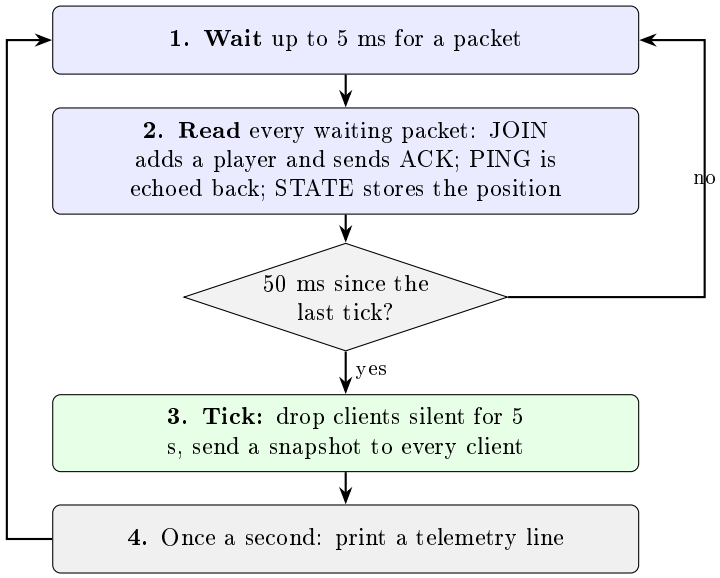


Figure 3: Main loop of the C++ server. It repeats forever on one thread.

5 How Both Servers Work: The Ins and Outs

TODO: write how the two servers work and how they differ.

	C++ UDP server	Godot ENet server
Networking	raw UDP datagrams	ENet through Godot's multiplayer peer
Default port	7777	7778
Tick timing	polls the clock every few ms and restarts its timer after each tick (about 19 ticks per second)	Godot timer that fires every 50 ms (about 20 ticks per second)
Join	JOIN message, ACK with player id	connection handled by ENet; the peer id is the player id
Position update	18-byte STATE datagram	remote procedure call (unreliable)
Snapshot	16 + 12N bytes, split into datagrams of at most 1,200 B	8 + 12N bytes (binary format), one remote procedure call
Client eviction	silent for 5 s	silent for 5 s
Latency measure	dedicated PING message echoed by the server	game client's ping: last send until next snapshot
Telemetry	one line per second	periodic server statistics

Table 3: The two backends side by side.

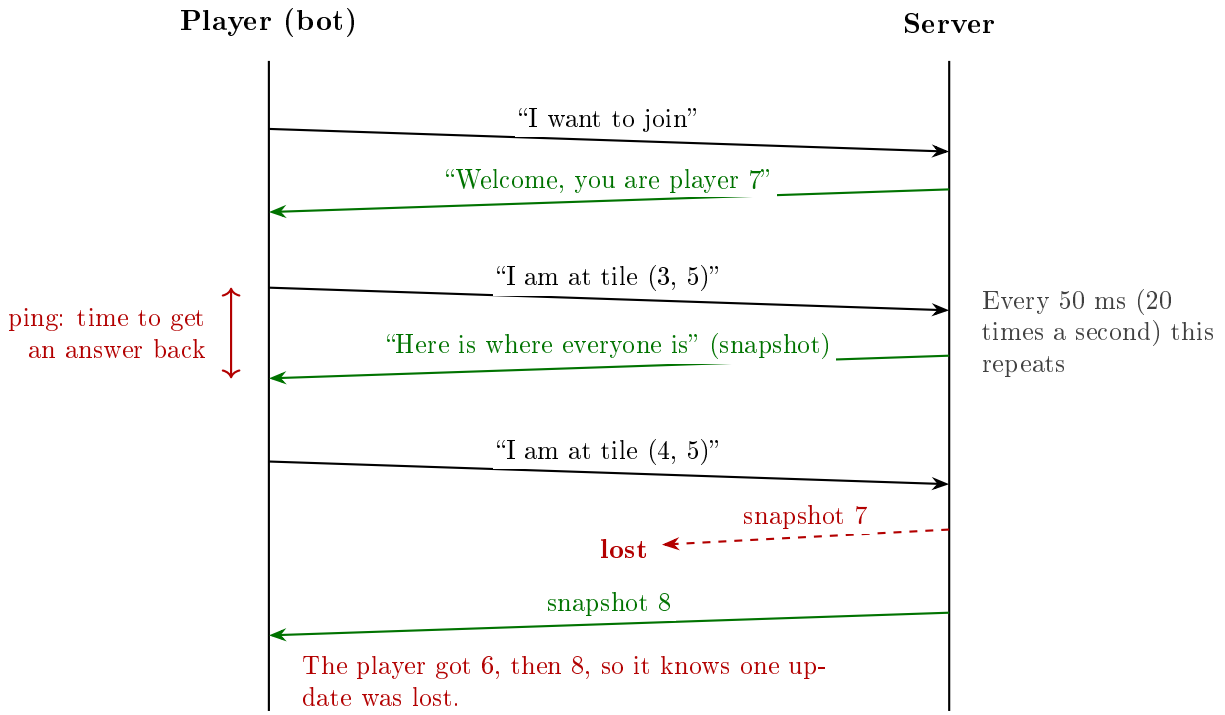


Figure 4: What the player and the server say to each other. The player says where it is, the server replies with where everyone is, and this repeats 20 times a second. Ping is how long it takes to hear back; a lost reply shows up as a missing number. The ENet backend does the same thing through Godot's own networking.

5.1 One tick, step by step

Every 50 ms the server does the same five things, in this order. If steps 1 to 4 take less than 50 ms, it waits for the next tick. If they take longer, the next tick starts late.

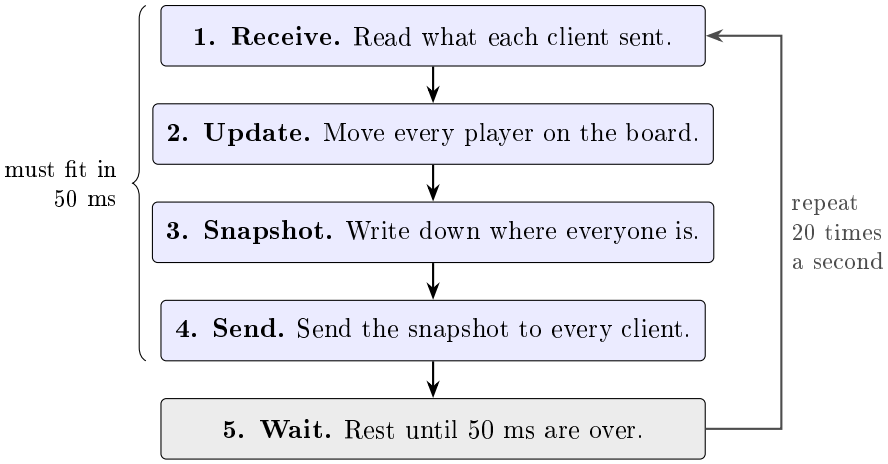


Figure 5: The server tick. Steps 1 to 4 are the work; step 5 is spare time. The more clients there are, the longer steps 3 and 4 take.

5.2 Why more clients means much more work

Each snapshot lists every player, and every client gets one. So when the number of clients N doubles, each snapshot gets about twice as big and there are twice as many to send: the work grows about four times. Figure 6 shows the amount of data this means for the C++ server (snapshot of $16 + 12N$ bytes sent to N clients, 20 times a second).

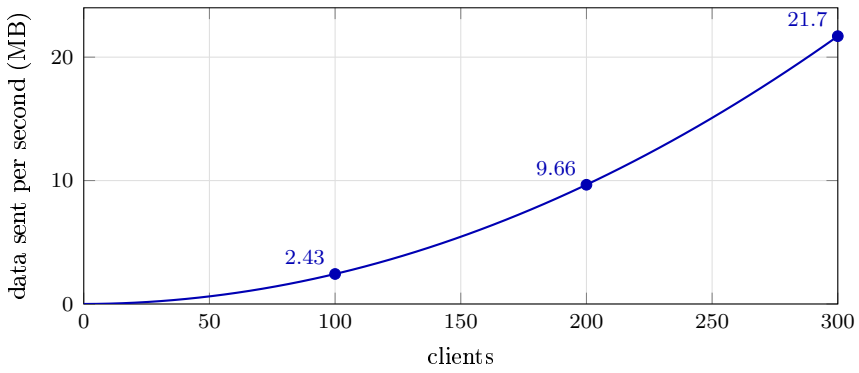


Figure 6: Data the server must send each second as clients are added (calculated from the packet format; ENet is similar). Tripling the clients from 100 to 300 multiplies the data about nine times.

5.3 How clients end up responding less (and recovering)

A client counts as *responding* if it received at least one new snapshot during the measurement period. While the server keeps up, every client gets a snapshot every tick. Once the work per tick is too much, snapshots arrive late, get skipped, or pile up behind each other, and some clients stop receiving any. Figure 7 shows the chain of events, and Figure 8 shows what one client sees.

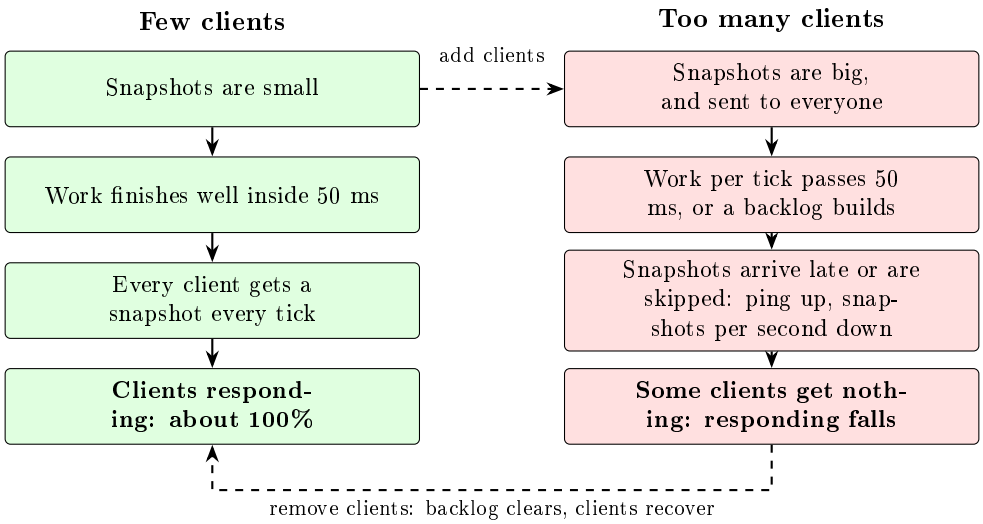


Figure 7: Left: a server that keeps up. Right: how adding clients leads to fewer clients responding. The dashed arrows show that removing load reverses it.

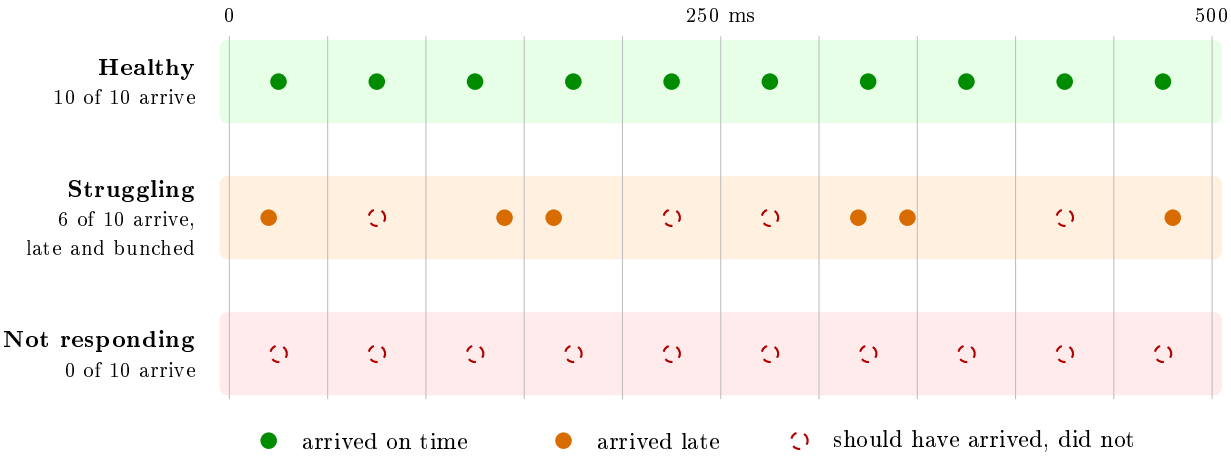


Figure 8: What one client receives over 500 ms (schematic). Each dot is a snapshot the server should send every 50 ms. Healthy: all arrive on time. Struggling: some come late and bunched together, others never arrive. Not responding: none arrive, so the client is counted as not responding.

6 Experimental Method

All runs use one server VM and three client VMs on Google Cloud (e2-standard-2 machines). ENet snapshots use the binary format from 22 September onward.

6.1 Main measures

Three measures are the main focus of the study. The rest of the data (CPU, memory, network use, server counters and every other column collected) is kept and still matters: it is used to explain why the main measures change. A tick-rate drop is the warning sign that one of the three is failing.

Measure	Plain meaning	On the charts
Latency	How long data takes to go to the server and back, in milliseconds (ping). Lower is better.	Ping in milliseconds (log scale)
Packet handling	How many packets the server receives and sends each second without losing or delaying them. A healthy client gets one snapshot per tick, about 20 per second.	Snapshots received per client per second (20 is perfect)
Client limit	The largest number of clients the server supports before performance degrades.	Share of clients still responding; where the lines break
Tick rate (warning sign)	Ticks the server completes per second. 20 Hz is healthy; below 18 Hz means the server is falling behind.	Server ticks per second, with an 18 Hz line

Table 4: The three main measures, and the tick-rate warning sign. All other collected data supports them.

6.2 Two kinds of test

We run two kinds of test, because they answer different questions.

- **Progressive (granular) ladder.** Start with one client and add clients one step at a time, holding each step for a short time (10 s in the October runs). The state list is 1 client, then 10, 11, 12, up to 300: 292 states per backend. Question: at what client count does the server stop keeping up?
- **Hold.** Ramp up to a target client count (targets 180, 260, 270 and 300), stop adding clients, and keep them connected for 20 minutes, measured in 120 windows of 10 s. Question: does the server stay stable at a fixed load?

6.3 How the data is collected

Figure 10 shows where a test runs. A Python script automates every run:

1. It creates one server VM and three client VMs on Google Cloud.
2. It starts the server (the C++ UDP server, or the Godot ENet server in headless mode).

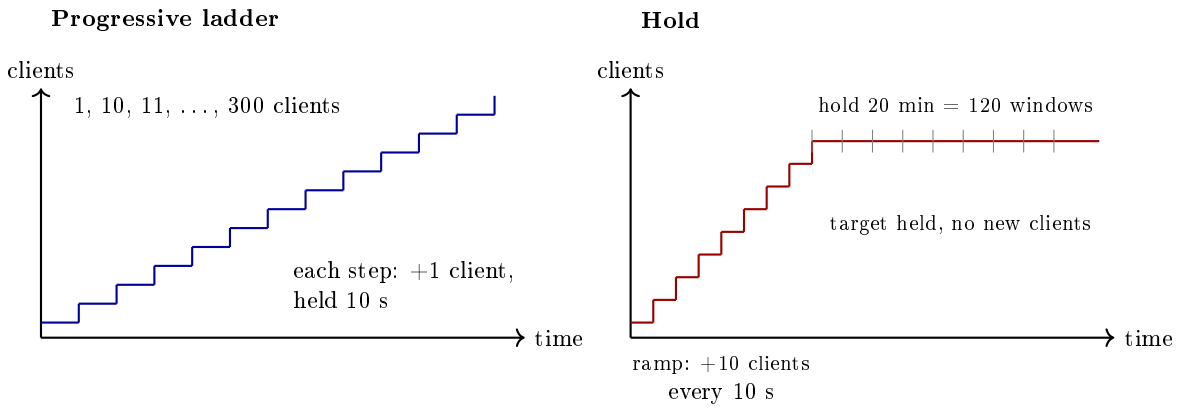


Figure 9: The two kinds of test (schematic). The ladder keeps adding clients and shows where performance breaks. The hold stops at a target and shows whether performance stays steady.

3. It gives each client VM a plan: a list of states, each with a total client target and a duration. The client VMs start bot clients so that the total matches the plan, and each bot plays the NodeStrike game.

4. During every state, each client counts what it sent and received, checks the snapshot sequence numbers for gaps, and measures ping about once per second.

5. The server writes one telemetry row per second (tick count, players connected, packets and bytes in and out). Each VM also records CPU, memory and network use every second.

6. When the run ends, all files are downloaded to this computer, the VMs are deleted, and analysis scripts merge the repetitions and apply pass/fail thresholds.

Gaps and incomplete snapshots are indicators of protocol-level loss, not measured network packet-loss percentages.

6.4 What the charts show

Every chart plots the same four things for both backends:

- **Latency:** average ping.
- **Packet handling:** snapshots received per client per second.
- **Clients responding:** how many clients are still getting answers.
- **Tick rate:** server ticks per second.

On the ladder charts the horizontal axis is the number of clients, so each point is one client count. On the hold charts it is time, so each point is one 10 s window.

6.5 Health criteria and caveats

TODO: write the pass/fail limits and what the measurements can and cannot show.

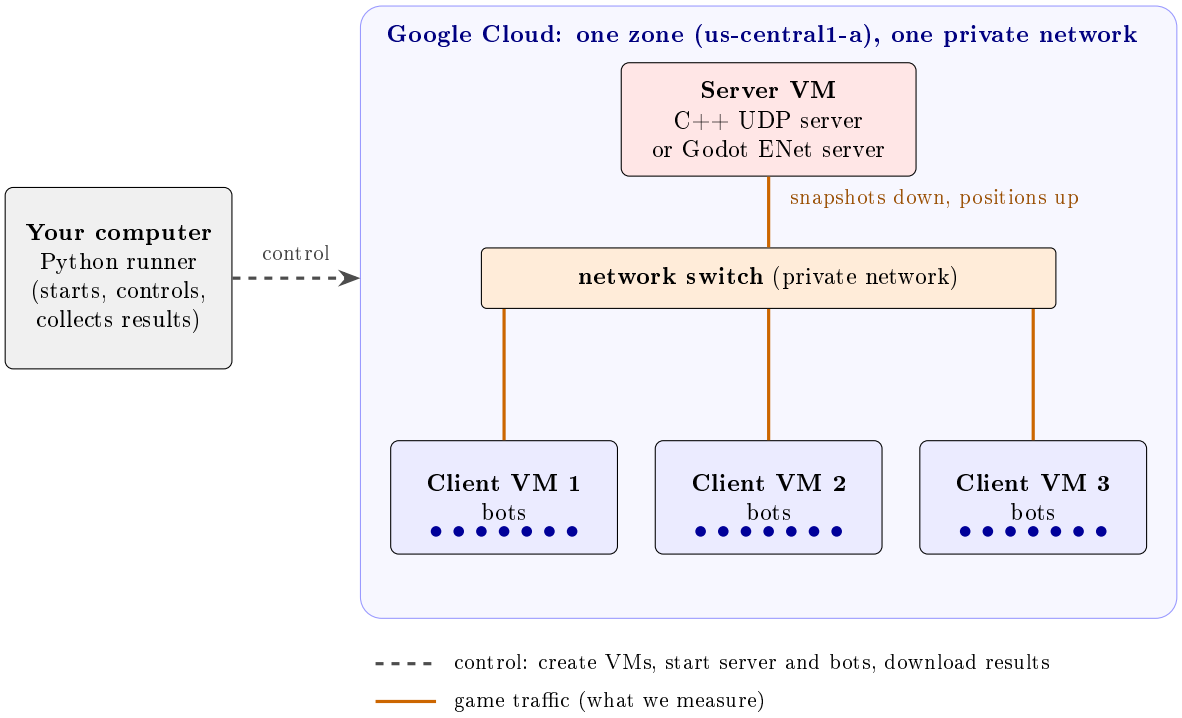


Figure 10: Where a test runs. The runner on your computer creates one server VM and three client VMs, starts the server, tells each client VM how many bots to run, and downloads the results afterwards. The bots and the server talk to each other over the cloud's private network (a switch). Each client VM runs about a third of the bots, so the server sees all of them as one crowd.

6.6 Runs

Granular ramps with $N_{\max} = 300$ give 292 states per backend (1 plus 10 through 300).

Date (2026)	Type	Backend	Counts	Time/state	Runs	Status
Oct 16–17	Progressive ladder	C++, ENet	1, 10–300	10 s	8 runs	ok
Oct 17–19	Hold	C++, ENet	180, 260, 270, 300	20 min hold	8 runs	ok

Table 5: Runs used in the results. The progressive ladder is eight complete runs per backend, each run a separate pass from 1 to 300 clients (one repetition each). Each hold target is also eight runs; the ENet captures that runs 3 and 4 were missing were repeated in a separate make-up batch.

7 Results

7.1 Test 1, progressive ladder: full run

Figure 11 is the progressive ladder test (Section 6). Clients are added one at a time from 1 to 300, so each point is one client count. Each line is the mean of 8 complete runs, and the shaded band shows the lowest and highest run.

Why the C++ tick rate is below 20 Hz. The C++ server checks the clock every few milliseconds and starts each tick a little after 50 ms have passed. Each tick is slightly late, so it completes about 19 ticks per second instead of 20. This is a small built-in delay, not overload, and the line stays above the 18 Hz threshold.

Figure 12 counts the ticks of both servers side by side, Figure 13 is the code behind it and Figure 14 explains it in pseudocode. The loop starts a tick as soon as it notices that 50 ms have passed, and then restarts its timer from that moment. If it notices 2 ms late, the next 50 ms are counted from the late moment, so the delay is never made up and each tick is 52 ms after the last. In 1 s that is $1000/52 \approx 19.2$ ticks instead of 20. In the measured data the tick rate falls from about 19.4 Hz (a 51.5 ms tick, about 1.5 ms late) with few clients to about 18.9 Hz (a 52.9 ms tick, about 2.9 ms late) at 300 clients, because with more clients each pass through the loop does more work, so the check for the next tick comes later.

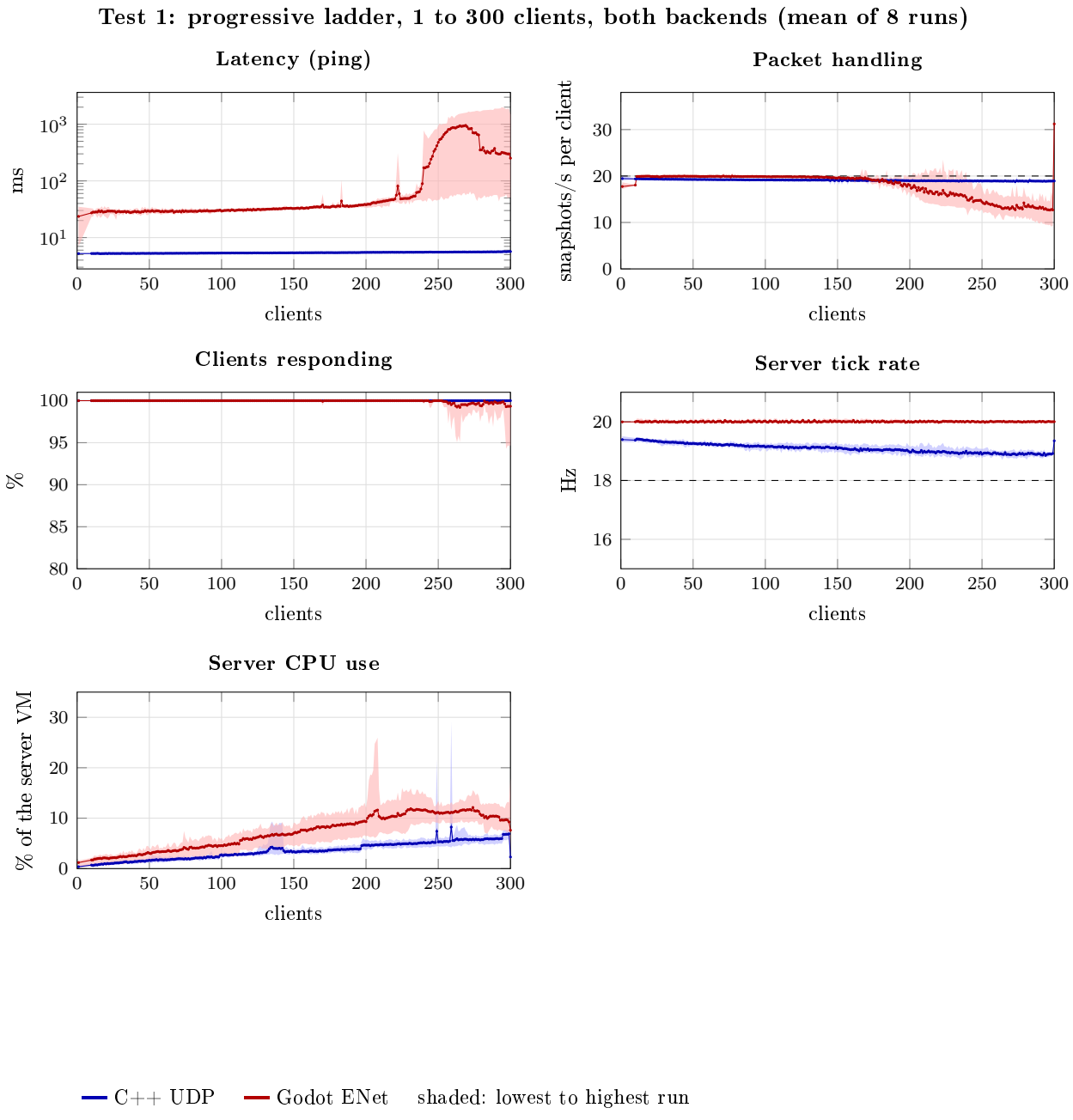


Figure 11: Test 1, progressive ladder (16–17 October, 10 s per state). Each line is the mean of 8 complete runs and the shaded band spans the lowest to the highest run. Ping is on a log scale. Dashed lines: 20 snapshots per second (one per tick) and the 18 Hz threshold. Server CPU use is the mean over each state, read from the server VM. ENet values above 20 snapshots per second cannot be real, since the server ticks at 20 Hz; they appear to come from delayed log reads under heavy load.

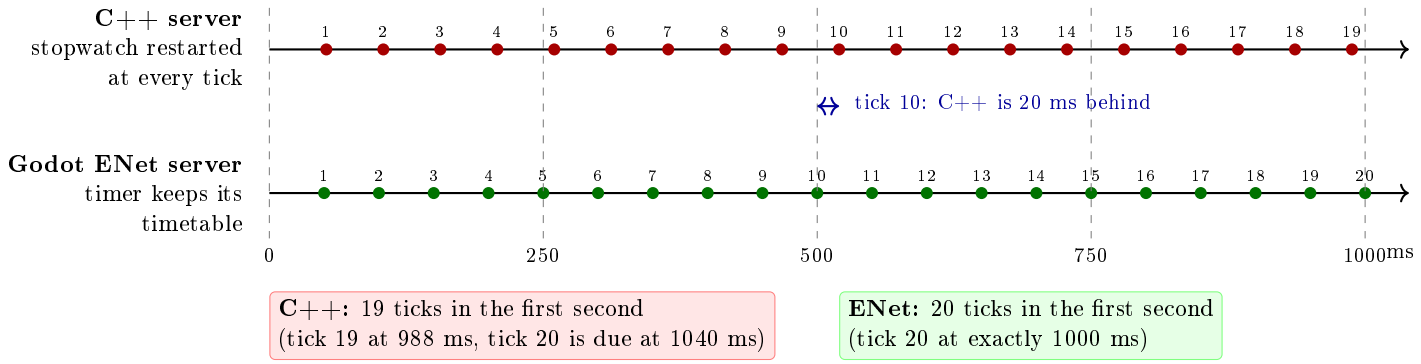


Figure 12: Counting ticks, C++ against ENet (schematic, each tick noticed 2 ms late). Every dot is one tick and the number is the tick count. The C++ loop restarts its stopwatch at each tick, so its ticks are 52 ms apart and it has counted only 19 ticks after one second. The Godot timer keeps its 50 ms timetable, so its tick 20 arrives at exactly 1000 ms. After 10 ticks the C++ server is already 20 ms behind, and the gap keeps growing.

```
// current code (server/main.cpp): the tick loop
auto   now     = Clock::now();
double elapsed = std::chrono::duration<double>(now - last_tick).count();
if (elapsed >= TICK_SEC) {      // 50 ms or more have passed
    last_tick = now;            // restart the timer at "now": the delay is lost
    tick_n++;
    // ... drop silent clients, broadcast snapshots ...
}

// fix (not applied): advance the timer by exactly one tick
last_tick += std::chrono::duration_cast<Clock::duration>(
                 std::chrono::duration<double>(TICK_SEC));
```

Figure 13: The tick check in the C++ server and the one-line change that would remove the drift.

```
STOPWATCH (the C++ server today)     TIMETABLE (Godot timer / the fix)
-------------------------------     ---------------------------------
every loop:                          every loop:
  if now - last_tick >= 50 ms:         if now - last_tick >= 50 ms:
    last_tick = now                      last_tick = last_tick + 50 ms
    do the tick                          do the tick

Stopwatch: noticed at 52 ms -> next tick due at 52 + 50 = 102 ms (drifts).
Timetable: noticed at 52 ms -> next tick due at 50 + 50 = 100 ms (on schedule).
```

Figure 14: Pseudocode for the two ways of timing a tick. Think of a bus that should leave every 50 minutes: the stopwatch driver restarts his watch whenever he notices the bus is due, so any lateness is kept; the timetable driver follows fixed times, so lateness is not kept.

7.2 Test 2, hold at 300 clients: ramp-up and hold

Figure 15 is the hold test at 300 clients, the heaviest target, summed up over 8 runs. The grey left part is the ramp-up, when clients are being added (plus a short wait); time 0 is the moment

the hold starts and no more clients are added. What matters is whether the lines recover after the ramp-up. C++ is steady in every run. ENet starts to fall behind during the ramp-up and does not recover: during the hold on average about 75% of its clients respond (61 to 88% across runs), it delivers about 16 snapshots per client per second, and its ping is around 1.4 s.

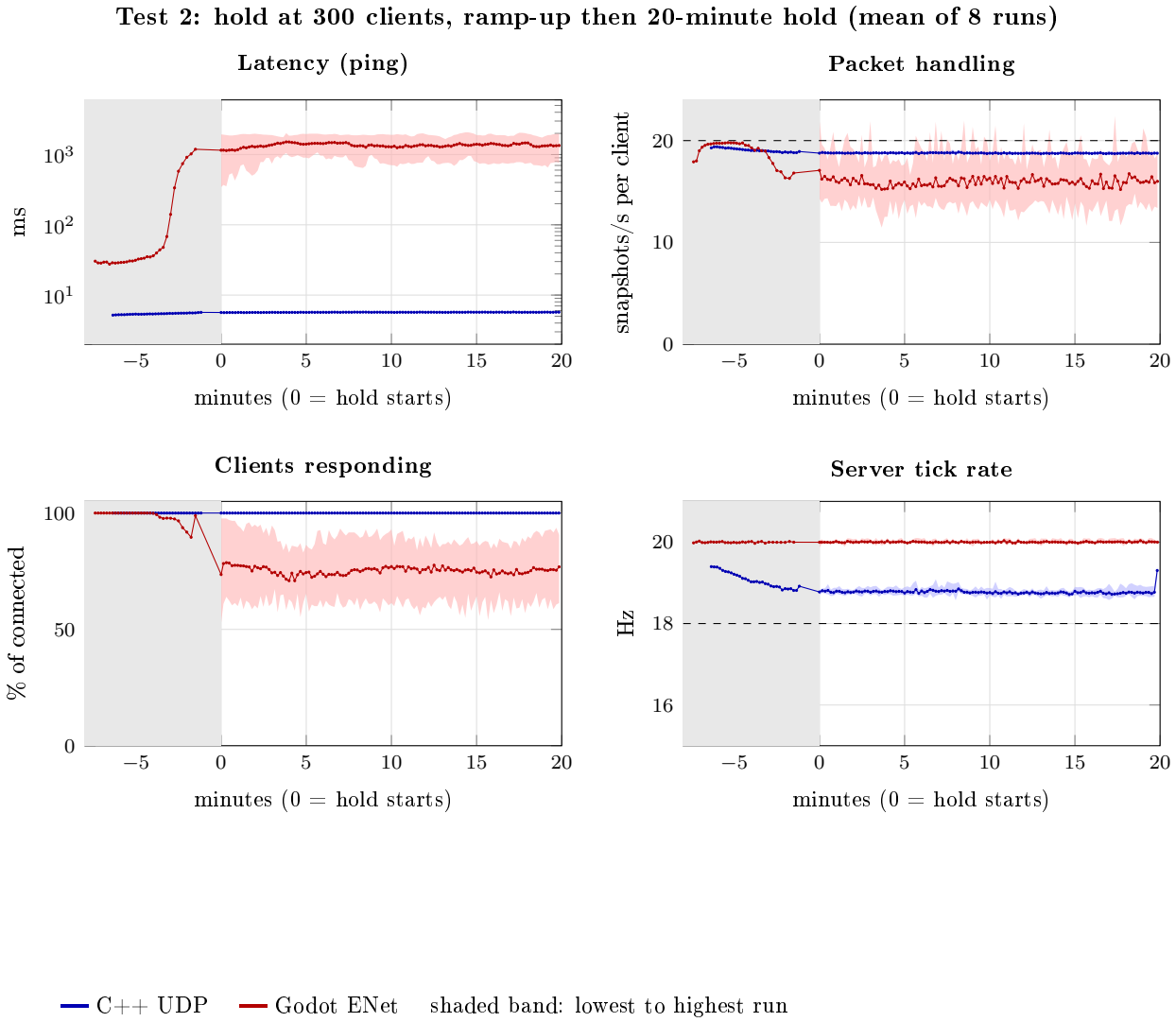


Figure 15: Test 2, hold at 300 clients, one point per 10 s. Each line is the mean of 8 runs and the band spans the lowest to the highest run. Grey: clients being added (ramp-up and a short wait before the hold). White: the 20-minute hold. Dashed lines: 20 snapshots per second and the 18 Hz threshold. ENet latency is the game client's logged ping; C++ latency is a raw round trip, so the two are not directly comparable (see Section 6).

7.3 Test 2, hold at every target

Figure 16 repeats the hold test for all four targets, again with the ramp-up on the left, and Table 6 sums up each target. C++ held every target in all eight runs. ENet held 180 clients in all eight runs, but only one of eight runs at 260 clients and one of eight at 270, and none at 300.

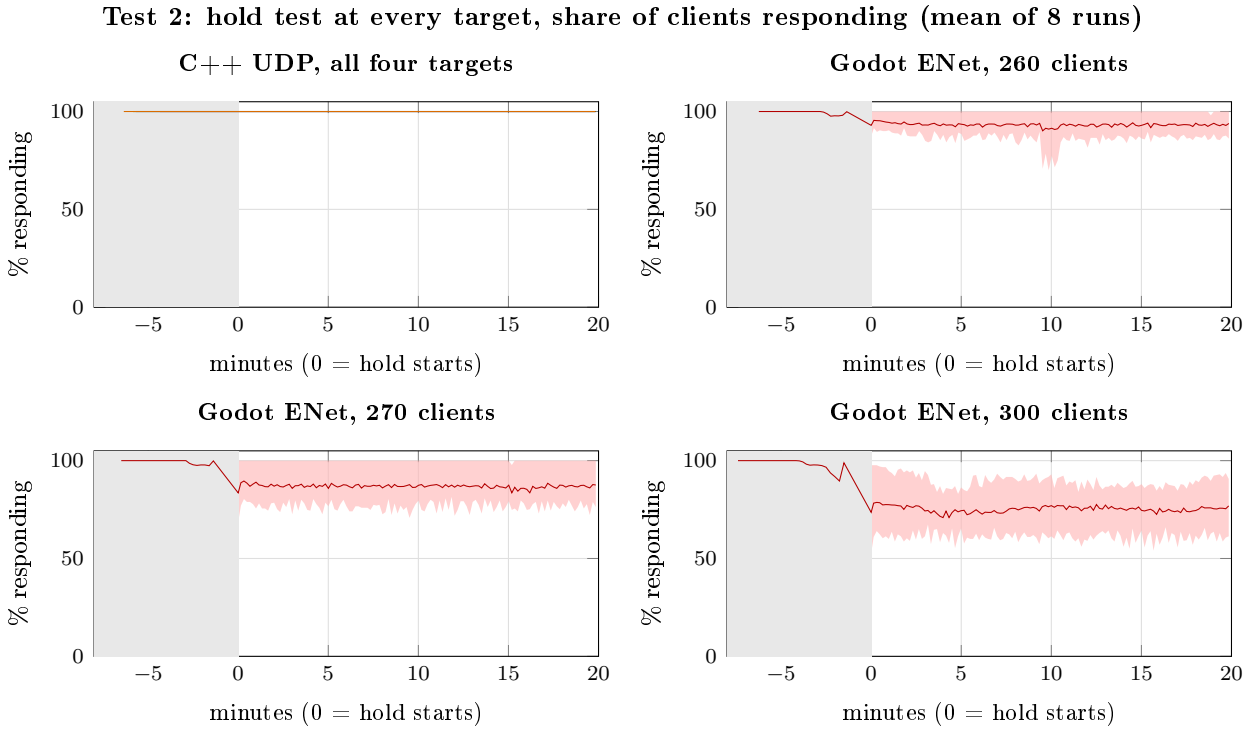


Figure 16: Test 2, share of clients responding during the hold at every target (mean of 8 runs). Grey: ramp-up. Top left: C++ UDP at 180, 260, 270 and 300 clients stays at 100% (all four lines overlap). The other panels show Godot ENet at 260, 270 and 300 clients; the line is the mean of 8 runs and the red band spans the lowest to the highest run. ENet at 180 clients also stays at 100% in all 8 runs and is not shown.

Backend	Clients	Resp. (%)	Snap/s	Ping, ms	Tick (Hz)	Held
C++ UDP	180	100.0 (100.0–100.0)	19.0	5 (5–6)	19.0	8 of 8
C++ UDP	260	100.0 (100.0–100.0)	18.8	6 (6–6)	18.8	8 of 8
C++ UDP	270	100.0 (100.0–100.0)	18.8	6 (6–6)	18.8	8 of 8
C++ UDP	300	100.0 (100.0–100.0)	18.8	6 (6–6)	18.8	8 of 8
Godot ENet	180	100.0 (100.0–100.0)	20.0	36 (35–36)	20.0	8 of 8
Godot ENet	260	93.2 (87.8–100.0)	19.3	701 (106–1267)	20.0	1 of 8
Godot ENet	270	86.9 (76.7–100.0)	18.5	927 (113–1612)	20.0	1 of 8
Godot ENet	300	75.2 (61.1–88.4)	15.9	1356 (924–1908)	20.0	0 of 8

Table 6: Hold results at each target, summed up over 8 runs. Each run is first averaged over its 20-minute hold; the table gives the mean of those averages with the lowest and highest run in brackets. Resp. = clients responding, Snap/s = snapshots received per client per second, Tick = server ticks per second. A run "held" if its hold averages met the health limits (at least 99% responding, at least 18 snapshots per second, ping at most 150 ms, tick at least 18 Hz). C++ ping is a raw round trip and ENet ping is the game client's ping, so they are not directly comparable.

7.4 Example data

The tables below are real rows from our runs, to show what the data looks like.

Progressive ladder: 8 runs summed up. Each row sums up the 8 complete ladder runs at one client count: the mean, with the lowest and highest run in brackets. Resp. = clients responding (mean), Snap/s = snapshots received per client per second, Tick = server ticks per second, Passing = runs that met the health limits at that count.

Backend	Clients	Resp. (%)	Ping, ms	Snap/s	Tick (Hz)	Passing
C++ UDP	100	100.0	5 (5–5)	19.1 (19.1–19.3)	19.1	8 of 8
C++ UDP	150	100.0	5 (5–5)	19.1 (19.0–19.2)	19.1	8 of 8
C++ UDP	200	100.0	5 (5–6)	19.0 (18.8–19.1)	19.0	8 of 8
C++ UDP	250	100.0	6 (5–6)	18.9 (18.8–19.1)	18.9	8 of 8
C++ UDP	300	100.0	6 (6–6)	18.9 (18.8–19.0)	19.3	8 of 8
Godot ENet	100	100.0	29 (28–31)	19.9 (19.8–19.9)	20.0	8 of 8
Godot ENet	150	100.0	33 (31–34)	19.6 (19.3–19.9)	20.0	8 of 8
Godot ENet	200	100.0	38 (34–42)	18.0 (16.5–20.2)	20.0	3 of 8
Godot ENet	250	100.0	478 (46–942)	14.4 (12.8–17.6)	20.0	0 of 8
Godot ENet	300	99.3	252 (50–1474)	31.2 (28.2–35.8)	20.0	3 of 8

Table 7: Example ladder rows, summed up over 8 runs (16–17 October, 10 s per state). Ping and Snap/s show the mean and, in brackets, the lowest and highest run. C++ ping is a raw round trip and ENet ping is the game client's ping, so they are not directly comparable.

Two things to note. First, the two backends measure latency differently: for C++ it is a raw round trip from a dedicated ping message, while for ENet it is the game client's logged ping (time until the next snapshot). They are not directly comparable until the Method section states how each is handled. Second, the ENet server's tick rate stays near 20 Hz even when its clients are failing, so for ENet the breakdown shows up as lost clients, low snapshot rate and high ping rather than a tick-rate drop. TODO: confirm why. The ENet server counts ticks from a Godot timer that

fires every 50 ms without drift, so its tick counter keeps pace with the clock whether or not the work finished in time.

7.5 How the runs were summed up

Both tests combine 8 complete runs per backend. For the ladder, each of the 292 client counts is paired with the same count in the other runs. For the holds, each 10 s state is paired by its position in the ramp-up or hold. The mean, lowest and highest value of each measure are reported. A ladder run passes at a client count if that count met the health limits; a hold run held if its hold averages met them. Hold runs 3 and 4 were missing some ENet captures because those captures failed, so they were repeated in a separate make-up batch and used in their place (ENet 270 and 300 clients for run 3, ENet 300 clients for run 4).

8 Discussion

TODO.

9 Conclusion

TODO.

References

[1] L. Aukstakalnis. *Server-Client Networking Implementation Using the Godot Game Engine*. 2024.

[2] F. T. Al-Dhief et al. Performance comparison between TCP and UDP protocols in different simulation scenarios. *International Journal of Engineering & Technology*, 7(4.36):172–176, 2018.

[3] C. Jones. Go UDP programming: a beginner-friendly guide to building fast, real-time apps. DEV Community, 2025.

[4] E. Retunsky. Benchmarking low-level I/O: C, C++, Rust, Golang, Java, Python. Star Gazers, 2023.

[5] Godot Foundation. Exporting for dedicated servers. *Godot Engine Documentation*, 2026.

[6] Godot Foundation. UDPServer. *Godot Engine Documentation*, 2026.

[7] Unity Technologies. Ticks and update rates. *Netcode for GameObjects 2.5 manual*. `https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/learn/ticks-and-update-rates.html`

[8] Valve Developer Community. Source Multiplayer Networking. `https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking`